SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems.* *(BL3)*

Assessment Tool Weightage: 40%

Dr M. Ramamoorthy

QUESTIONS: **Total Marks:40**

Annexure A

TECHNICAL PRESENTATION

Code of Conduct:

I (K . Deepak / 192320078) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

TECHNICAL PRESENTATION

1. Object-Oriented Design in Smart Healthcare Systems

Introduction

A Smart Healthcare System is used to manage patients, doctors, appointments, and medical records. Object-Oriented Design (OOD) helps organize the system into different classes and objects, making it easier to develop and maintain.

Explanation

Class

A class is a blueprint for creating objects.

Examples:

- Patient
- Doctor
- Appointment
- Medical Record

Each class has its own data and functions.

Object

An object is a real example of a class.

Examples:

- Patient: Rahul
- Doctor: Dr. Priya
- Appointment: A101

Objects store information and perform actions.

Encapsulation

Encapsulation keeps data safe by hiding it from unauthorized users.

Example:

Only doctors can update a patient's medical record.

Inheritance

Inheritance allows one class to use the properties of another class.

Example:

A Person class can be the parent of Patient and Doctor classes.

This avoids writing the same code again.

Polymorphism

Polymorphism allows one method to perform different tasks.

Example:

The method DisplayDetails() shows different information for a Patient and a Doctor.

Analysis

- The system is easy to expand by adding new modules like Pharmacy or Billing.
- Patient information is secure because data is protected.
- Changes in one module do not affect other modules.
- Code can be reused, which saves development time.
- The system becomes more reliable and easier to maintain.

Solution / Recommendation

- Create separate classes for Patient, Doctor, Appointment, Billing, and Medical Record.
- Use encapsulation to protect patient data.
- Use inheritance to reduce duplicate code.
- Use polymorphism to perform different tasks with the same method.
- Follow modular design so new features can be added easily.

Conclusion

Object-Oriented Design makes the Smart Healthcare System secure, flexible, and easy to maintain. It improves software quality, reduces development time, and supports future improvements.

2. Applying Encapsulation and Abstraction in Online Banking

Introduction

An Online Banking System allows customers to perform activities such as checking account balances, transferring money, paying bills, and viewing transaction history. Object-oriented concepts like **abstraction** and **encapsulation** help make the banking system secure, easy to use, and simple to maintain.

Explanation

Abstraction

means showing only the necessary features to the user while hiding the complex internal process. For example, when a customer transfers money, they only enter the account details and amount. The customer does not see how the bank verifies the account, checks the balance,

processes the transaction, or updates the database. All these internal operations are hidden, making the system simple and user-friendly.

Encapsulation

means keeping data and methods together in a class and protecting sensitive information from unauthorized access. In an online banking system, details such as account number, password, PIN, and account balance are kept private. These details can only be accessed or modified through authorized methods after proper authentication. This prevents unauthorized users from changing important information.

Comparison

Although both concepts hide information, they serve different purposes. Abstraction focuses on hiding unnecessary implementation details to simplify the system for users, while encapsulation focuses on protecting data by restricting direct access. Abstraction improves usability, whereas encapsulation improves data security.

Analysis

Using abstraction makes the banking application easier for customers because they only interact with simple options like fund transfer or balance enquiry. Encapsulation ensures that sensitive customer information remains safe and prevents accidental or unauthorized modifications. Together, these concepts improve the reliability, security, and maintainability of the banking system. They also make future updates easier without affecting the users.

Solution / Recommendation

To build a secure online banking system, developers should use abstraction to simplify banking operations and encapsulation to protect customer information. Strong authentication methods such as passwords and OTP verification should also be used to improve security. Regular software updates and proper access control should be implemented to keep the system reliable.

Conclusion

Abstraction and encapsulation are essential concepts in object-oriented design. Abstraction makes the banking system simple and easy to use, while encapsulation protects sensitive customer data. Together, they help develop a secure, reliable, and efficient online banking system.

3. Modeling Object Relationships in a University Management System

Introduction

A University Management System is used to manage students, faculty, departments, and courses. Object-oriented relationship modeling helps represent how these objects are connected. The three main relationships used are **association, aggregation, and composition**. These relationships make the system more organized and easier to maintain.

Explanation

Association is a relationship where two objects are connected but can exist independently. For example, a Student enrolls in a Course. Even if the student leaves the university, the course still exists, and if a course is discontinued, the student still exists. This is a simple relationship between two independent objects.

Aggregation is a "has-a" relationship where one object contains another, but both can exist independently. For example, a Department has many Faculty Members. If a department is closed, the faculty members can still work in another department. Therefore, both objects can exist separately.

Composition is a stronger form of aggregation where one object completely depends on another. For example, a Student has a Student ID Card. If the student record is deleted, the ID card also becomes invalid because it cannot exist without the student. This is called composition.

Analysis

Using these relationships improves the design of the University Management System. Association helps connect different objects without making them dependent on each other. Aggregation allows objects to be reused in different parts of the system. Composition ensures that closely related objects are managed together. These relationships reduce code duplication, improve flexibility, and make the system easier to update and maintain. They also help developers understand the system structure more clearly.

Solution / Recommendation

To develop an efficient University Management System, association should be used for student-course relationships, aggregation for department-faculty relationships, and composition for student-ID card or student-profile relationships. Choosing the correct relationship for each object improves software quality, reduces maintenance effort, and makes future modifications easier.

Conclusion

Object relationships are an important part of object-oriented design. Association, aggregation, and composition help organize the University Management System effectively. They improve flexibility, code reusability, maintainability, and overall software performance.

4. Importance of SDLC in E-Commerce Development

Introduction

The Software Development Life Cycle (SDLC) is a structured process used to develop high-quality software. In an E-Commerce platform, it helps build features like product management, online payments, and order tracking efficiently.

Explanation

SDLC includes six phases: Requirement Analysis, Design, Coding, Testing, Deployment, and Maintenance. Requirements are collected first, then the system is designed and developed. Testing removes errors before deployment, and maintenance keeps the software updated and secure.

Analysis

Each phase improves software quality and reliability. Proper planning reduces errors, testing ensures smooth performance, and maintenance helps add new features and fix bugs. This increases customer satisfaction and reduces development risks.

Solution / Recommendation

Follow every SDLC phase carefully, perform regular testing, protect customer data with security measures, and maintain the system by fixing bugs and updating features.

Conclusion

SDLC helps develop a secure, reliable, and user-friendly E-Commerce platform. It improves software quality, reduces risks, and ensures long-term customer satisfaction.

5. Selecting an Appropriate SDLC Model for Mobile App Development

Introduction

A company is developing a Mobile Learning Application where customer requirements change frequently. Choosing the right SDLC model is important for successful development.

Explanation

The **Waterfall Model** follows a fixed sequence of phases and is suitable when requirements are clear and do not change. The **Agile Model** develops the application in small iterations and allows frequent customer feedback and changes. The **Spiral Model** combines development with risk analysis and is mainly used for large and complex projects.

Analysis

The Waterfall Model is simple but does not handle changing requirements well. The Spiral Model is flexible but is costly and takes more time. The Agile Model is the best choice because it quickly adapts to changing requirements, improves customer satisfaction, and delivers updates continuously.

Recommendation

The **Agile Model** is the most suitable for a Mobile Learning Application because it supports continuous feedback, faster development, and easy modification of features whenever customer requirements change.

Conclusion

The Agile Model provides flexibility, better teamwork, and high-quality software, making it the best SDLC model for mobile app development.

6. Modular Design in a Hospital Management System

Introduction

A Hospital Management System may face problems if all its modules are tightly connected. This makes the system difficult to maintain and update.

Explanation

When modules are tightly coupled, a change in one module can affect the entire system. By using object-oriented principles such as **encapsulation, inheritance, and modular design**, separate modules like Patient, Doctor, Billing, and Pharmacy can work independently.

Analysis

Modular design makes the system easier to maintain because changes in one module do not affect others. It also improves code reusability, reduces errors, and allows new features to be added without changing the whole system. This increases the overall efficiency and scalability of the software.

Solution / Recommendation

The system should be divided into independent modules such as Patient Management, Appointment Management, Billing, Pharmacy, and Medical Records. Each module should have its own responsibilities and interact with other modules only when required.

Conclusion

Modular design improves maintainability, scalability, and code reusability. It makes the Hospital Management System more reliable, flexible, and easier to manage.

7. Enhancing Software Quality Using Design Patterns

Introduction

A Hotel Reservation System may face problems such as duplicate code and inefficient object creation. Design patterns help solve these problems and improve software quality.

Explanation

The **Factory Pattern** creates different types of booking objects without exposing the creation process. The **Singleton Pattern** ensures that only one instance of a class, such as the database connection, exists throughout the application. The **Observer Pattern** automatically notifies users about booking confirmations, cancellations, or room availability updates.

Analysis

Using design patterns reduces duplicate code, improves flexibility, and makes the software easier to maintain. They also improve system performance and simplify future updates by following a standard design approach.

Solution / Recommendation

The Hotel Reservation System should use the **Factory Pattern** for creating booking objects, the **Singleton Pattern** for managing database connections, and the **Observer Pattern** for sending booking notifications. This makes the system more efficient and reliable.

Conclusion

Design patterns improve software quality by reducing code duplication, increasing flexibility, and making the Hotel Reservation System easier to maintain and expand.

8. UML-Based Modeling for an Online Examination System

Introduction

An Online Examination System is used to manage students, instructors, examinations, and results. UML (Unified Modeling Language) helps in designing and understanding the system before development.

Explanation

The **Use Case Diagram** shows how users such as students and instructors interact with the system. The **Class Diagram** represents classes like Student, Instructor, Exam, and Result along with their attributes and methods. The **Activity Diagram** shows the flow of activities, such as student login, taking the exam, submitting answers, and viewing results.

Analysis

UML diagrams provide a clear view of the system structure and workflow. They help developers understand requirements, identify errors at an early stage, and improve communication among developers, clients, and other stakeholders. This reduces development time and improves software quality.

Solution / Recommendation

The development team should prepare UML diagrams before coding. This helps in planning the system properly, avoiding design errors, and making future modifications easier.

Conclusion

UML modeling improves system visualization, requirement analysis, and communication. It helps develop a well-structured, reliable, and easy-to-maintain Online Examination System.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness